

EPLB

Deep Dive into DeepSeek — Chapter 28

Yin Yang

Foundation Books Project

The one rule of this chapter

Placement may change *where* a logical expert runs.

It must **never** change *which* logical expert a routed token asked for.

- A router's histogram can look balanced while a deployment still puts two hot experts on the same GPU.
- EPLB sits between the routing record and the distributed execution plan: it duplicates hot logical experts and assigns physical replicas to GPUs and nodes.
- Every replica of a logical expert must still compute that expert's learned function — replication is a deployment decision, not a new model choice.

The one rule of this chapter

Placement may change *where* a logical expert runs.

It must **never** change *which* logical expert a routed token asked for.

- A router's histogram can look balanced while a deployment still puts two hot experts on the same GPU.
- EPLB sits between the routing record and the distributed execution plan: it duplicates hot logical experts and assigns physical replicas to GPUs and nodes.
- Every replica of a logical expert must still compute that expert's learned function — replication is a deployment decision, not a new model choice.

Goal: follow one four-expert load vector through balanced packing, redundant replicas, hierarchical and global policy, the public interface, and profile drift — and see where each object lives in the EPLB source.

1. Router balance is not placement balance

Roadmap

1. Router balance is not placement balance
2. Load, placement, and packing objects

Roadmap

1. Router balance is not placement balance
2. Load, placement, and packing objects
3. Redundant experts and policy selection

1. Router balance is not placement balance
2. Load, placement, and packing objects
3. Redundant experts and policy selection
4. The public interface

1. Router balance is not placement balance
2. Load, placement, and packing objects
3. Redundant experts and policy selection
4. The public interface
5. Profiles, drift, and adjacent research

Roadmap

1. Router balance is not placement balance
2. Load, placement, and packing objects
3. Redundant experts and policy selection
4. The public interface
5. Profiles, drift, and adjacent research
6. Measurement, validation, and the arc closes

**Router balance is not placement
balance**

Same histogram, different device loads

Quantity	Value	Role
$w = (w_0, w_1, w_2, w_3)$	$(70, 60, 50, 40)$	one-layer load vector, running example
total load	220	sum of w_e
ideal per-GPU share	$220/4 = 55$	target if load splits perfectly

naive:

GPU 0
 $F_0, F_0: 70$

GPU 1
 $F_1, F_1: 60$

GPU 2
 $F_2, F_2: 50$

GPU 3
 $F_3, F_3: 40$

Same histogram, different device loads

Quantity	Value	Role
$w = (w_0, w_1, w_2, w_3)$	$(70, 60, 50, 40)$	one-layer load vector, running example
total load	220	sum of w_e
ideal per-GPU share	$220/4 = 55$	target if load splits perfectly

naive:

GPU 0
 $F_0, F_0: 70$

GPU 1
 $F_1, F_1: 60$

GPU 2
 $F_2, F_2: 50$

GPU 3
 $F_3, F_3: 40$

balanced:

GPU 0
 $F_0, F_3: 55$

GPU 1
 $F_0, F_3: 55$

GPU 2
 $F_2, F_1: 55$

GPU 3
 $F_2, F_1: 55$

Same histogram, different device loads

Quantity	Value	Role
$w = (w_0, w_1, w_2, w_3)$	(70, 60, 50, 40)	one-layer load vector, running example
total load	220	sum of w_e
ideal per-GPU share	$220/4 = 55$	target if load splits perfectly

naive:

GPU 0
 $F_0, F_0: 70$

GPU 1
 $F_1, F_1: 60$

GPU 2
 $F_2, F_2: 50$

GPU 3
 $F_3, F_3: 40$

balanced:

GPU 0
 $F_0, F_3: 55$

GPU 1
 $F_0, F_3: 55$

GPU 2
 $F_2, F_1: 55$

GPU 3
 $F_2, F_1: 55$

Nothing about the router changed between these two rows. Only the physical placement changed.

Load, placement, and packing objects

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts
Placement map ϕ	physical slot $\rightarrow$ logical expert, shape $L \times P$

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts
Placement map ϕ	physical slot $\rightarrow$ logical expert, shape $L \times P$
Inverse map	logical expert $\rightarrow$ valid physical slots, shape $L \times E \times X$

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts
Placement map ϕ	physical slot $\rightarrow$ logical expert, shape $L \times P$
Inverse map	logical expert $\rightarrow$ valid physical slots, shape $L \times E \times X$
Replica count r_e	$ \{p : \phi(p) = e\} $, must agree with the inverse map

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts
Placement map ϕ	physical slot $\rightarrow$ logical expert, shape $L \times P$
Inverse map	logical expert $\rightarrow$ valid physical slots, shape $L \times E \times X$
Replica count r_e	$ \{p : \phi(p) = e\} $, must agree with the inverse map

If every expert is duplicated once, the modeled per-replica loads are $w_e/r_e = 35, 30, 25, 20$ for F_0, F_1, F_2, F_3 .

The objects a placement plan is built from

Object	Definition
Load matrix W	nonnegative $L \times E$ matrix, layers by logical experts
Placement map ϕ	physical slot $\rightarrow$ logical expert, shape $L \times P$
Inverse map	logical expert $\rightarrow$ valid physical slots, shape $L \times E \times X$
Replica count r_e	$ \{p : \phi(p) = e\} $, must agree with the inverse map

If every expert is duplicated once, the modeled per-replica loads are $w_e/r_e = 35, 30, 25, 20$ for F_0, F_1, F_2, F_3 .

The load matrix says how much work exists. The placement and inverse maps say where it may legally run. Measured traffic is a third, separate object that neither one supplies.

The recurring subproblem: pack, don't just count

Item weights 35, 35, 30, 30, 25, 25, 20, 20; four packs; two items per pack.

- Sort items high to low; place each item into the currently lightest pack that still has room.

The recurring subproblem: pack, don't just count

Item weights 35, 35, 30, 30, 25, 25, 20, 20; four packs; two items per pack.

- Sort items high to low; place each item into the currently lightest pack that still has room.
- Poor pairing (35, 35), (30, 30), (25, 25), (20, 20): loads (70, 60, 50, 40).

The recurring subproblem: pack, don't just count

Item weights 35, 35, 30, 30, 25, 25, 20, 20; four packs; two items per pack.

- Sort items high to low; place each item into the currently lightest pack that still has room.
- Poor pairing (35, 35), (30, 30), (25, 25), (20, 20): loads (70, 60, 50, 40).
- Greedy pairing (35, 20), (35, 20), (30, 25), (30, 25): loads (55, 55, 55, 55).

The recurring subproblem: pack, don't just count

Item weights 35, 35, 30, 30, 25, 25, 20, 20; four packs; two items per pack.

- Sort items high to low; place each item into the currently lightest pack that still has room.
- Poor pairing (35, 35), (30, 30), (25, 25), (20, 20): loads (70, 60, 50, 40).
- Greedy pairing (35, 20), (35, 20), (30, 25), (30, 25): loads (55, 55, 55, 55).

`balanced_packing(weight, num_packs)` is the one routine that both the hierarchical and the global policy call, at every level of the placement hierarchy.

Redundant experts and policy selection

Extra slots go to the highest load-per-copy

Every logical expert starts with one physical copy. Each remaining slot goes to $\arg \max_e W_e/r_e$, then r_e increments.

Step	Winner	Score before win	Replica counts after
1	F_0	70/1	(2, 1, 1, 1)

Extra slots go to the highest load-per-copy

Every logical expert starts with one physical copy. Each remaining slot goes to $\arg \max_e W_e/r_e$, then r_e increments.

Step	Winner	Score before win	Replica counts after
1	F_0	70/1	(2, 1, 1, 1)
2	F_1	60/1	(2, 2, 1, 1)

Extra slots go to the highest load-per-copy

Every logical expert starts with one physical copy. Each remaining slot goes to $\arg \max_e W_e/r_e$, then r_e increments.

Step	Winner	Score before win	Replica counts after
1	F_0	70/1	(2, 1, 1, 1)
2	F_1	60/1	(2, 2, 1, 1)
3	F_2	50/1	(2, 2, 2, 1)

Extra slots go to the highest load-per-copy

Every logical expert starts with one physical copy. Each remaining slot goes to $\arg \max_e W_e/r_e$, then r_e increments.

Step	Winner	Score before win	Replica counts after
1	F_0	70/1	(2, 1, 1, 1)
2	F_1	60/1	(2, 2, 1, 1)
3	F_2	50/1	(2, 2, 2, 1)
4	F_3	40/1	(2, 2, 2, 2)

Extra slots go to the highest load-per-copy

Every logical expert starts with one physical copy. Each remaining slot goes to $\arg \max_e W_e/r_e$, then r_e increments.

Step	Winner	Score before win	Replica counts after
1	F_0	70/1	(2, 1, 1, 1)
2	F_1	60/1	(2, 2, 1, 1)
3	F_2	50/1	(2, 2, 2, 1)
4	F_3	40/1	(2, 2, 2, 2)

Redundancy budget $R_{\text{extra}} = P - E$. A redundant expert is an additional physical copy, never a new learned function.

One divisibility test, two placement properties

Policy	Selected when	What it preserves
Hierarchical	<code>num_groups mod num_nodes = 0</code>	group-to-node locality
Global	otherwise (one group, one node)	device-load balance only

- Running example: $G_0 = \{F_0, F_1\}$, $G_1 = \{F_2, F_3\}$, two nodes. Hierarchical GPU loads (65, 65, 45, 45), ratio $65/55 \approx 1.18$.

One divisibility test, two placement properties

Policy	Selected when	What it preserves
Hierarchical	<code>num_groups mod num_nodes = 0</code>	group-to-node locality
Global	otherwise (one group, one node)	device-load balance only

- Running example: $G_0 = \{F_0, F_1\}$, $G_1 = \{F_2, F_3\}$, two nodes. Hierarchical GPU loads (65, 65, 45, 45), ratio $65/55 \approx 1.18$.
- Global packing of the same replicas reaches (55, 55, 55, 55), ratio 1, but no longer proves group locality.

One divisibility test, two placement properties

Policy	Selected when	What it preserves
Hierarchical	$\text{num_groups} \bmod \text{num_nodes} = 0$	group-to-node locality
Global	otherwise (one group, one node)	device-load balance only

- Running example: $G_0 = \{F_0, F_1\}$, $G_1 = \{F_2, F_3\}$, two nodes. Hierarchical GPU loads (65, 65, 45, 45), ratio $65/55 \approx 1.18$.
- Global packing of the same replicas reaches (55, 55, 55, 55), ratio 1, but no longer proves group locality.
- On the pinned twelve-expert fixture the two policies disagree on *which* experts get the extra replica in layer 1 — policy choice changes replica selection, not only packing.

One divisibility test, two placement properties

Policy	Selected when	What it preserves
Hierarchical	$\text{num_groups} \bmod \text{num_nodes} = 0$	group-to-node locality
Global	otherwise (one group, one node)	device-load balance only

- Running example: $G_0 = \{F_0, F_1\}$, $G_1 = \{F_2, F_3\}$, two nodes. Hierarchical GPU loads (65, 65, 45, 45), ratio $65/55 \approx 1.18$.
- Global packing of the same replicas reaches (55, 55, 55, 55), ratio 1, but no longer proves group locality.
- On the pinned twelve-expert fixture the two policies disagree on *which* experts get the extra replica in layer 1 — policy choice changes replica selection, not only packing.

Neither branch predicts inter-node traffic or throughput; both return the same three tensors under the same invariants.

The public interface

rebalance_experts: five inputs, three outputs

Object	Role
weight	$L \times E$ estimated load, converted to FP32 on CPU
num_replicas, num_groups, num_nodes, num_gpus	topology and budget
phy2log	logical expert stored in each physical slot
log2phy	physical slots holding each logical expert
logcnt	replica count per logical expert

rebalance_experts: five inputs, three outputs

Object	Role
weight	$L \times E$ estimated load, converted to FP32 on CPU topology and budget
num_replicas,	
num_groups,	
num_nodes, num_gpus	
phy2log	logical expert stored in each physical slot
log2phy	physical slots holding each logical expert
logcnt	replica count per logical expert

EPLB is an off-hot-path CPU planner: ordinary PyTorch control flow, not a token-path kernel. It returns candidate maps; a deployment still has to load weights, activate the map, and validate it.

From concept to code: the policy branch

The divisibility test and the CPU planning boundary are both literally in the entry point.

```
from EPLB/eplb.py

weight = weight.float().cpu()
if num_groups % num_nodes == 0:
    # use hierarchical load-balance policy
    phy2log, phyrank, logcnt = rebalance_experts_hierarchical(
        weight, num_replicas, num_groups, num_nodes, num_gpus)
else:
    # use global load-balance policy
    phy2log, phyrank, logcnt = rebalance_experts_hierarchical(
        weight, num_replicas, 1, 1, num_gpus)
```

From concept to code: the policy branch

The divisibility test and the CPU planning boundary are both literally in the entry point.

```
from EPLB/eplb.py

weight = weight.float().cpu()
if num_groups % num_nodes == 0:
    # use hierarchical load-balance policy
    phy2log, phyrank, logcnt = rebalance_experts_hierarchical(
        weight, num_replicas, num_groups, num_nodes, num_gpus)
else:
    # use global load-balance policy
    phy2log, phyrank, logcnt = rebalance_experts_hierarchical(
        weight, num_replicas, 1, 1, num_gpus)
```

The global branch is not a different algorithm — it is the same hierarchical routine called with one group and one node, which is why both branches return maps satisfying the same replica-count invariant. [INSPECTED: EPLB eplb.py]

Profiles, drift, and adjacent research

What EPLB owns, and what a deployment still has to build

EPLB owns

- Balanced packing, redundant-expert allocation, hierarchical and global policy.
- The three returned tensors and their consistency invariant.

A deployment still owns

- Load prediction, profiling cadence, and drift detection.
- Live weight migration, request draining, atomic cutover, and rollback.

What EPLB owns, and what a deployment still has to build

EPLB owns

- Balanced packing, redundant-expert allocation, hierarchical and global policy.
- The three returned tensors and their consistency invariant.

A deployment still owns

- Load prediction, profiling cadence, and drift detection.
- Live weight migration, request draining, atomic cutover, and rollback.
- A placement valid for yesterday's profile can be poor for tomorrow's traffic mix — that is workload drift, not a bug in the old placement.
- LPLB is adjacent research: a GPU-resident LP solver that redirects *current-batch* assignments among an already-redundant topology. It is early-stage, not a measured successor to EPLB.

What EPLB owns, and what a deployment still has to build

EPLB owns

- Balanced packing, redundant-expert allocation, hierarchical and global policy.
- The three returned tensors and their consistency invariant.

A deployment still owns

- Load prediction, profiling cadence, and drift detection.
- Live weight migration, request draining, atomic cutover, and rollback.
- A placement valid for yesterday's profile can be poor for tomorrow's traffic mix — that is workload drift, not a bug in the old placement.
- LPLB is adjacent research: a GPU-resident LP solver that redirects *current-batch* assignments among an already-redundant topology. It is early-stage, not a measured successor to EPLB.

The released repository plans; it does not migrate, drain, or roll back.

Measurement, validation, and the arc closes

Balance and locality are two different questions

Placement	Max/mean	Modeled inter-node share
Hierarchical (toy)	1.18	0

Balance and locality are two different questions

Placement	Max/mean	Modeled inter-node share
Hierarchical (toy)	1.18	0
Global (toy)	1.00	0.50

Balance and locality are two different questions

Placement	Max/mean	Modeled inter-node share
Hierarchical (toy)	1.18	0
Global (toy)	1.00	0.50

- Better load balance can move more traffic across nodes — the two metrics trade off, and neither one is a latency measurement by itself.
- A reviewed two-node, four-H200 replay confirms exact planner execution and deterministic hashes; its source-localized runs show hierarchical placement keeping traffic local while global placement does not, in a trace deliberately constructed to expose that contrast.

Balance and locality are two different questions

Placement	Max/mean	Modeled inter-node share
Hierarchical (toy)	1.18	0
Global (toy)	1.00	0.50

- Better load balance can move more traffic across nodes — the two metrics trade off, and neither one is a latency measurement by itself.
- A reviewed two-node, four-H200 replay confirms exact planner execution and deterministic hashes; its source-localized runs show hierarchical placement keeping traffic local while global placement does not, in a trace deliberately constructed to expose that contrast.

The synthetic round-trip timings are logical-payload numbers, not DeepEP production traffic or hardware fabric counters. [OBSERVED: two-node four-H200 EPLB/DualPipe replay, reviewed]

Buys

- A candidate replication and placement plan from a declared load vector, in ordinary CPU control flow.
- Two coherent policies — locality-preserving or globally balanced — behind one divisibility test.
- Three consistent tensors a runtime can consume.

Does not buy

- Load prediction — EPLB starts from a load vector, it does not estimate one.
- A live handoff: weight migration, request draining, atomic cutover, rollback.
- A measured throughput or latency result.

Closing the arc

Buys

- A candidate replication and placement plan from a declared load vector, in ordinary CPU control flow.
- Two coherent policies — locality-preserving or globally balanced — behind one divisibility test.
- Three consistent tensors a runtime can consume.

Does not buy

- Load prediction — EPLB starts from a load vector, it does not estimate one.
- A live handoff: weight migration, request draining, atomic cutover, rollback.
- A measured throughput or latency result.

EPLB decides what the physical placement *is*. The runtime and orchestration chapters decide how a deployment gets there safely.

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`
- **group-limited routing config context** → `DeepSeek-V3/inference/configs/config_671B.json`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`
- **group-limited routing config context** → `DeepSeek-V3/inference/configs/config_671B.json`
- **DeepEP monitoring counter feeding a profile** → `DeepEP/deep_ep/buffers/elastic.py`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`
- **group-limited routing config context** → `DeepSeek-V3/inference/configs/config_671B.json`
- **DeepEP monitoring counter feeding a profile** → `DeepEP/deep_ep/buffers/elastic.py`
- **adjacent LP-based redirection research** → `LPLB/lplb/planner.py`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`
- **group-limited routing config context** → `DeepSeek-V3/inference/configs/config_671B.json`
- **DeepEP monitoring counter feeding a profile** → `DeepEP/deep_ep/buffers/elastic.py`
- **adjacent LP-based redirection research** → `LPLB/lplb/planner.py`
- **dependency-free placement reference and tests** →
`code/chapter_eplb/eplb_placement_reference.py`

Concept-to-code map for the chapter

- **balanced packing, redundant experts, policy branch** → `EPLB/eplb.py`
- **public README example and interface overview** → `EPLB/README.md`
- **group-limited routing config context** → `DeepSeek-V3/inference/configs/config_671B.json`
- **DeepEP monitoring counter feeding a profile** → `DeepEP/deep_ep/buffers/elastic.py`
- **adjacent LP-based redirection research** → `LPLB/lplb/planner.py`
- **dependency-free placement reference and tests** →
`code/chapter_eplb/eplb_placement_reference.py`

Replicate hot experts, pack them onto devices, keep the logical route intact.

Next: the DualPipe chapter consumes the resulting communication constraints while preserving the logical route and the active placement generation.